

Job Market Analytics Platform

Technical Implementation and Architecture

Author

June 19, 2026

Contents

1	Core Platform & Infrastructure Topology	7
1.1	Overview	7
1.2	Core Platform Topology	7
1.3	Next.js Production Containerization	10
1.4	Database Health Probing & Startup Sequencing	10
1.5	Internal Docker DNS Service Discovery	11
1.6	Environment Variable Strategy	11
1.7	Elasticsearch Configuration	12
1.8	Technology Choice: Docker Compose vs Kubernetes	12
2	Data Acquisition Pipeline (Web Scraping)	14
2.1	Overview	14
2.2	Scraping Architecture and CI/CD Integration	14
2.2.1	CI/CD Dual-Runtime Pipeline	15
2.2.2	Technical Flow	17
2.3	Playwright vs Traditional Scrapers	17
2.3.1	Technology Choice: Playwright vs Alternatives	18
2.3.2	Anti-Scraping Countermeasures	18
2.4	Concurrency Control and Data Freshness	21
2.5	Data Normalization Pipeline	21
2.5.1	Phase 1: Pattern-Based Cleansing	22
2.5.2	Phase 2: Semantic Normalization and Feature Extraction	22
2.5.3	Phase 3: Deterministic Content Hashing	23
2.5.4	Phase 4: Persistence	23
2.5.5	Normalization Flow Diagram	23
2.6	Key Quantitative Metrics	25

3	Polyglot Persistence and Search Infrastructure	26
3.1	Overview	26
3.2	Polyglot Persistence Justification	26
3.2.1	Why Not a Monolithic Database?	27
3.3	Supabase (Core Relational Database)	28
3.3.1	Structured Entities	28
3.3.2	Dual-Client Architecture	30
3.4	Redis (Session, Cache, and Security Layer)	30
3.4.1	Singleton Connection Pattern	30
3.4.2	Redis Key Space Architecture	30
3.4.3	Core Workloads	31
3.5	Elasticsearch (Full-Text Search Engine)	31
3.5.1	CQRS Data Flow	33
3.5.2	Data Synchronization	34
3.5.3	Advanced Mapping and Indexing	34
3.6	Technology Choice: Elasticsearch vs Alternatives	35
3.7	Key Quantitative Metrics	36
4	Full-Text Search Engine (Elasticsearch)	37
4.1	Search Architecture & Rationale	37
4.2	Index Schema & Mapping Strategies	38
4.3	Data Ingestion & Deduplication Pipeline (Sync Flow)	39
4.4	Vietnamese NLP & Heuristic Normalization Logic	41
4.5	Multi-Field Search & Relevance Tuning	42
4.6	Aggregation & Memory-Resident Enum Caching	44
4.7	Key Quantitative Metrics	46
5	Authentication & Authorization	47
5.1	Architecture Overview	47
5.1.1	Technology Choice: Supabase Auth vs Alternatives	48
5.2	Authentication Flow	49
5.3	Route Protection Strategy	51
5.4	Advanced Security: JWT Blacklisting via Redis	51
5.4.1	Optimized Storage Process:	52
5.4.2	Fail-Closed vs Fail-Open: Deliberate Security Trade-offs	54
5.5	Rate Limiting	54
5.5.1	Mechanism:	54
5.5.2	Design Decisions:	55

5.6	Form Management and Input Validation	55
5.7	Key Quantitative Metrics	56
6	Core Frontend Layer (Next.js)	57
6.1	Overview	57
6.1.1	Technology Choice: Next.js App Router vs Alternatives	58
6.2	SSR Data Fetching and User-Aware Rendering	59
6.3	Faceted Search Architecture	59
6.3.1	URL-Driven State Management	59
6.3.2	Supported Filter Parameters	61
6.4	Client-Side Vietnamese NLP Processing	61
6.4.1	Location and Geographic Parsing (<code>splitLocations</code>) .	61
6.4.2	Unstructured Salary Categorization (<code>getSalaryBuckets</code>)	62
6.4.3	Experience Tier Mapping (<code>getExpBuckets</code>)	62
6.5	Page Architecture	63
6.6	Key Quantitative Metrics	64

Chapter 1

Core Platform & Infrastructure Topology

1.1 Overview

The foundation of the CareerIntel platform is a highly resilient, polyglot microservices architecture. Designed to integrate web scraping, full-text search, and real-time user interfaces, the infrastructure relies on Docker Compose to orchestrate seven concurrent containers. This topology ensures that the Next.js frontend, the Elasticsearch search engine, and the core PostgreSQL database operate seamlessly within a secure, internal software-defined network.

This chapter focuses on the four core platform services that underpin the data management and user-facing layers (Next.js, Elasticsearch, Redis, MongoDB). The remaining three services `chatbot-api`, `celery-worker`, and `qdrantform` the AI Chatbot subsystem and are documented in the companion thesis (Thesis 1).

1.2 Core Platform Topology

The primary user-facing and data management layers of the application are supported by the following containerized services:

- **next-app**: The Next.js 16 (App Router) frontend serving both Server-Side Rendered (SSR) web pages and API routes (port 3000). It acts as

the primary gateway for users interacting with the job search interface and user profiles.

- **elasticsearch:** An Elasticsearch 8.13.0 node optimized for complex full-text queries and faceted filtering, essential for querying thousands of normalized job postings.
- **redis:** Operates as an in-memory session cache and handles rapid enumerations (e.g., job tracking state), utilizing the **ioredis** library on the Next.js side. The container is configured with **password-based authentication** (`-requirepass`) to prevent unauthorized access, and employs **Append-Only File (AOF) persistence** (`-appendonly yes`) to ensure that cached data survives container restarts by writing every mutation to an append-only log on disk.
- **mongodb:** Provides document-based storage for flexible data schemas, utilized heavily by the Next.js API routes for managing user chat sessions via the `chatService`.

(Note: The core user and job data are persisted in a managed Supabase PostgreSQL instance, which the containerized services access securely via Service Role Keys).

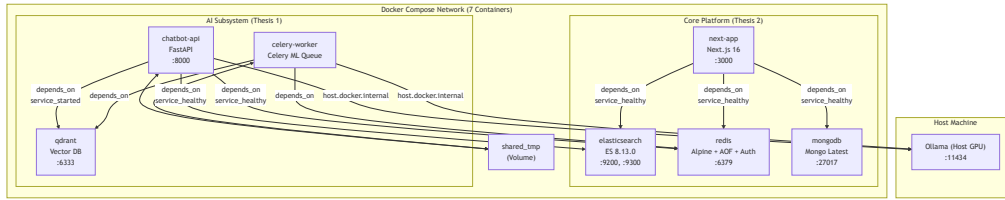


Figure 1.1: Docker Compose Network Topology (7 Containers)

1.3 Next.js Production Containerization

The `next-app` service is deployed using a highly optimized, multi-stage Dockerfile (38 lines) designed specifically for Node.js environments.

1. **Stage 1 (deps)**: Utilizes `node:20-alpine` and `npm ci` to cleanly install dependencies, maximizing cache hits during the build process.
2. **Stage 2 (builder)**: Compiles the React application. Telemetry is explicitly disabled (`NEXT_TELEMETRY_DISABLED=1`) to prevent external analytics requests during CI/CD. The build outputs an optimized `.next` standalone directory.
3. **Stage 3 (runner)**: The final production image. It explicitly copies only the five necessary compilation artifacts (`package.json`, `node_modules`, `public`, `.next`, `next.config.js`) from the builder stage. This drastically reduces the final attack surface and image size, establishing a secure environment configured tightly around `NODE_ENV=production`.

1.4 Database Health Probing & Startup Sequencing

In a distributed microservices environment, strict boot sequencing is mandatory to prevent cascade failures when upstream dependencies are unresponsive. CareerIntel implements comprehensive database health probing directly within the `docker-compose.yml`.

Services do not blindly depend on container startup; instead, they utilize the `depends_on` configuration coupled with `condition: service_healthy`.

Service	Health Check Command	Interval	Start Period
Redis	<code>redis-cli -a \${REDIS_PASSWORD} ping</code>	10s	10s
MongoDB	<code>mongosh -eval "db.adminCommand('ping')"</code>	15s	20s
Elasticsearch	<code>curl _cluster/health (status: green/yellow)</code>	30s	60s

Table 1.1: Database Health Check Configurations

The Next.js and FastAPI application containers are instructed by Docker Compose to halt their startup procedures until these specific health conditions evaluate to true, guaranteeing a robust boot process. Notably, the

Redis health check includes the authentication password flag (`-a`), verifying not just TCP connectivity but the full authentication layer. MongoDB's generous 20-second start period accommodates disk initialization latency, while Elasticsearch's 60-second start period allows time for JVM warmup and shard allocation.

1.5 Internal Docker DNS Service Discovery

To eliminate the fragility of hardcoded IP addresses, the architecture leverages Docker's internal DNS resolver. All containers reside within a default bridge network. Consequently, services map to each other using their logical container names:

- The Next.js API connects to the Redis session store natively via `redis://:${REDIS_PASSWORD}@redis`.
- The internal Elasticsearch sync scripts connect to the cluster via `http://elasticsearch:9200`.
- The Chatbot API connects to MongoDB via `mongodb://${MONGO_USER}:${MONGO_PASS}@mongodb`.

1.6 Environment Variable Strategy

The application employs a dual-file environment strategy to separate concerns:

- **.env.local**: Contains application-specific secrets (Supabase keys, API tokens) used by the Next.js application both during local development and within the Docker container.
- **.env.docker**: Contains Docker-specific infrastructure credentials (Redis password, MongoDB root credentials) referenced by `docker-compose.yml` for service configuration.

This separation ensures that infrastructure credentials are not leaked into the application runtime, and application secrets are not exposed to database containers.

1.7 Elasticsearch Configuration

The Elasticsearch container is explicitly configured for local development and integration within the microservices cluster. It is bootstrapped as a single node (`discovery.type=single-node`), removing the overhead of quorum elections. Furthermore, X-Pack security features are disabled (`xpack.security.enabled=false`) to streamline internal container-to-container communication without the friction of TLS and basic authentication management during the development phase. Finally, JVM memory limits are strictly enforced via environment variables (`ES_JAVA_OPTS=-Xms512m -Xmx512m`) to prevent the Java heap from starving other containers of host RAM, capping the Elasticsearch process at 1 GB total (512 MB initial + 512 MB maximum).

1.8 Technology Choice: Docker Compose vs Kubernetes

The platform employs Docker Compose for container orchestration rather than Kubernetes (K8s). While Kubernetes offers superior features for production-grade deployments including automatic horizontal pod autoscaling, self-healing via liveness/readiness probes, rolling updates with zero-downtime deployments, and declarative service mesh routing its operational complexity introduces significant overhead for a development-phase project of this scale.

Criteria	Docker Compose	Kubernetes
Configuration complexity	Single YAML file (145 lines)	Multiple resource manifests (Deployment, Service, ConfigMap, Ingress)
Learning curve	Minimal <code>docker compose up -d</code>	Requires <code>kubectl</code> , Helm charts, namespace management
Startup time	Seconds	Minutes (cluster bootstrapping, pod scheduling)
Health checks	Native <code>healthcheck</code> directive	Liveness/Readiness probes with finer granularity
Scaling	Manual (<code>docker compose scale</code>)	Automatic HPA based on CPU/memory metrics
Service discovery	Container name DNS (built-in)	CoreDNS with Service objects
Resource overhead	~100 MB for Docker Engine	~2 GB for control plane (etcd, API server, scheduler)

Table 1.2: Docker Compose vs Kubernetes Comparison

Docker Compose provides the ideal balance of simplicity and functionality for a seven-container development topology, while the architecture is designed to be Kubernetes-compatible for future production migration.

Chapter 2

Data Acquisition Pipeline (Web Scraping)

2.1 Overview

The Job Market Analytics Platform employs a robust, automated data acquisition pipeline designed to harvest, normalize, and persist job postings from major Vietnamese recruitment portals. The pipeline primarily targets TopCV and JobOKO, utilizing Playwright (headless Chromium) to handle JavaScript-heavy DOM rendering. This architecture is orchestrated via a GitHub Actions cron job running every three days (cron: `0 19 */3 * *`, corresponding to 02:00 AM Vietnam time), ensuring a continuous influx of fresh job data directly into the Supabase (PostgreSQL) data layer. The workflow also supports manual triggering via `workflow_dispatch` for on-demand data refresh.

2.2 Scraping Architecture and CI/CD Integration

The entry point for the data acquisition pipeline is `backend/jobs/github_action.ts`, which is triggered by the GitHub Actions cron schedule every three days. This automation ensures the platform's dataset remains current without requiring manual intervention.

2.2.1 CI/CD Dual-Runtime Pipeline

The GitHub Actions workflow orchestrates a multi-language pipeline, installing both **Node.js 22** (for the Playwright scrapers) and **Python 3.11** (for the offline normalization service) within a single CI run. The pipeline timeout is set to **150 minutes** to accommodate the sequential scraping of multiple sources with rate-limiting delays.

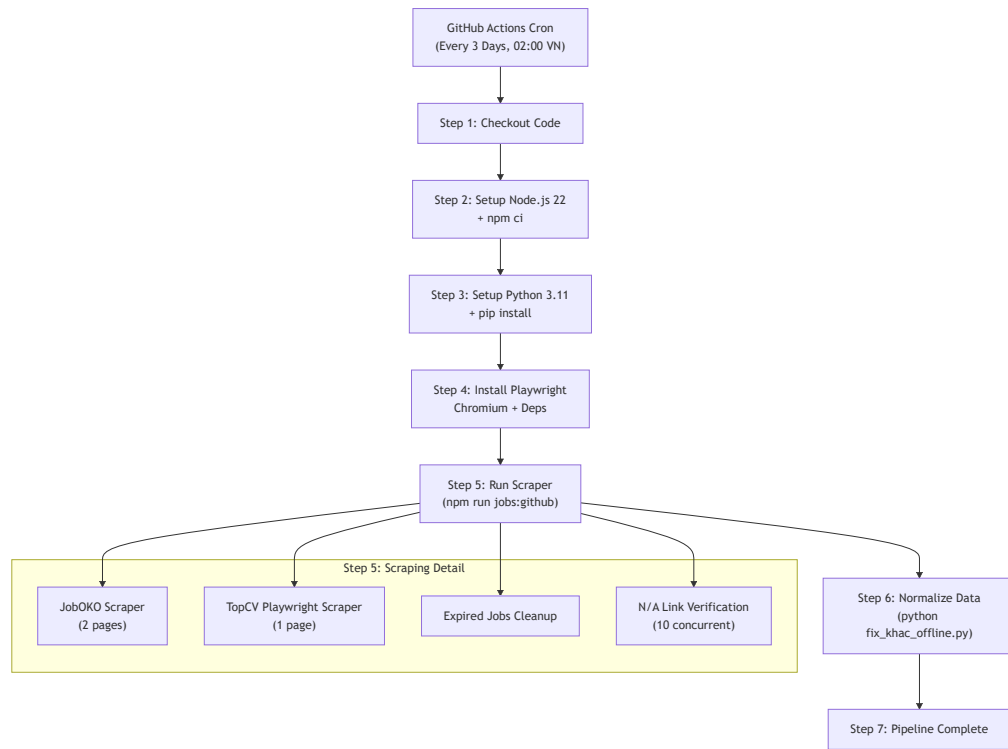


Figure 2.1: CI/CD Scraping Pipeline Workflow

2.2.2 Technical Flow

1. **Initialization:** The process begins by validating Supabase Service Role Keys, establishing a secure connection that bypasses Row Level Security (RLS) policies for administrative ingestion.
2. **Scraping Sources:**
 - **JobOKO** (`scrapeJoboko`): Configured dynamically via environment variables. The scraper delay is set to `3000ms` in the CI environment (via `SCRAPER_DELAY_MS`), though local development defaults to `7000ms` for more conservative rate limiting.
 - **TopCV** (`scrapeTopCV`): Employs a more sophisticated, two-pass scraping pattern using Playwright to handle dynamic content (323 lines of scraper logic).
3. **Direct Upsert:** Data extracted by the scrapers is immediately persisted into the Supabase `jobs` table using an `upsert` operation, keyed on the unique job `url`. To optimize CI runtimes, data normalization is intentionally bypassed during the scraping phase and deferred to an offline Python script (`fix_khac_offline.py`). This script runs within the same CI job but as a separate step, because the FastAPI ML Service is not started in the CI environment it is intended for local development or future production deployment.

2.3 Playwright vs Traditional Scrapers

Modern recruitment portals like TopCV rely heavily on client-side JavaScript for rendering job listings dynamically. Traditional scraping tools (such as BeautifulSoup or Scrapy) operate on raw HTML and frequently fail to parse these dynamic Document Object Models (DOM). To overcome this, the platform utilizes Playwright's headless Chromium browser (`-no-sandbox, -disable-setuid-sandbox`), which executes the JavaScript and evaluates the DOM only after it has been fully rendered.

2.3.1 Technology Choice: Playwright vs Alternatives

Criteria	Playwright	Puppeteer
JS rendering	Full Chromium engine	Full Chromium engine
Language support	Node.js, Python, Java, C#	Node.js only
Multi-browser	Chromium, Firefox, WebKit	Chromium only
CI/CD support	Native (<code>npx playwright install</code>)	Requires manual binary set
Auto-wait	Built-in smart waits	Manual <code>waitForSelector</code>
Headless performance	~200 MB per context	~200 MB per context

Table 2.1: Scraping Technology Comparison

Playwright was selected over Puppeteer for its superior multi-language support (enabling potential migration to Python scrapers in the future), native CI/CD integration, and built-in auto-wait mechanisms that simplify scraping JavaScript-rendered pages.

2.3.2 Anti-Scraping Countermeasures

To ensure reliable extraction and avoid IP bans, the `scrap_topcv.ts` implementation incorporates several evasion strategies:

- **User-Agent Spoofing:** The scraper simulates a legitimate user by mimicking a Chrome 120 browser operating on Windows 10 via the `User-Agent` header.
- **Accept-Language Header:** The scraper sets `Accept-Language: vi-VN,vi;q=0.9,en`, signaling to the target server that the request originates from a Vietnamese-language browser. This reduces the likelihood of being flagged by geographic anomaly detection systems.
- **Delay Injection:** Aggressive rate limiting is applied. The system introduces an 8-second delay before querying listing pages to allow dynamic content to fully render. Between individual detail page crawls, a fixed 10-second delay is enforced to prevent triggering rate-limit thresholds on the target server.
- **Two-Pass Scraping:**

- *Pass 1 (Listing)*: Extracts high-level metadata (title, URL, company name, salary, location, and logo) from search result pages, filtering out duplicate URLs in-memory using a JavaScript **Set**.

- *Pass 2 (Detail)*: Iterates over the deduplicated URLs, opens individual pages, waits for DOM readiness (2.5 seconds), and extracts deep content (full description, requirements, benefits, and expiration date) using heuristic DOM traversal techniques.

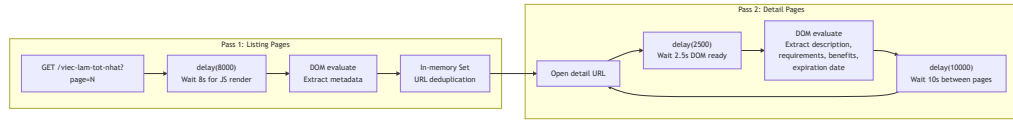


Figure 2.2: Two-Pass Scraping Flow for Dynamic Content

2.4 Concurrency Control and Data Freshness

Managing thousands of network requests requires strict concurrency limits to prevent connection timeouts and avoid overwhelming target servers.

- **Worker Pool Pattern (`runWithConcurrency`):** The pipeline utilizes a robust concurrency limit pattern, capping active workers at 10 concurrent requests for link checking. Workers independently request the next task from the pool upon completion, which is more efficient than batching with `Promise.all` and prevents isolated timeouts from stalling the entire queue. Each worker independently captures its own result (fulfilled or rejected), ensuring fault isolation.
- **Expired Jobs Cleanup:** To maintain data relevance, a maintenance job periodically fetches all records with explicit expiration dates (parsed from DD/MM/YYYY strings). These dates are compared against the current date, and expired records are purged in batches of 50 to minimize load on the Supabase Postgres instance. The batch size of 50 is chosen to comply with Supabase's `.in()` filter constraint, which limits the number of elements in a single filter predicate.
- **N/A Links Verification:** For job postings without an explicit expiration date (marked as 'N/A'), the system verifies the survival of the listing by dispatching HTTP requests to the URL via the `checkJobExists` function. Dead links are subsequently batch-deleted from the database in groups of 50.

2.5 Data Normalization Pipeline

While the primary scraping mechanism persists raw data directly to the database to optimize ingestion speed, the platform implements a robust, secondary offline processing pipeline to standardize and enrich this data. This normalization process is executed by a dedicated Python Machine Learning Service (334 lines), designed as a four-phase Extract, Transform, Load (ETL) pipeline.

2.5.1 Phase 1: Pattern-Based Cleansing

The initial phase addresses common structural anomalies inherent in scraped text. Utilizing targeted regular expressions, the system cleanses mandatory fields such as company names and geographical locations, stripping out irrelevant characters, errant whitespace, and inconsistent formatting to establish a baseline of clean string data.

2.5.2 Phase 2: Semantic Normalization and Feature Extraction

The core of the data enrichment process leverages a hybrid approach combining artificial intelligence with rule-based heuristics.

- **Job Title Standardization:** Regular expressions are applied to isolate the core professional title by removing promotional prefixes (e.g., "Tuyn gp," "Cn tuyn") and descriptive suffixes detailing salary or urgency.
- **Categorization:** The pipeline maps diverse job listings into **66 pre-defined industry domains** (defined in the `CORE_DOMAINS` dictionary, spanning from "An toàn lao ng" to "Sinh viên / Thc tp sinh"). It primarily utilizes the **Gemini 2.5 Flash** AI model to semantically categorize the job based on its description and context. To ensure reliability against API rate limits or service unavailability, a fallback rule-based system is implemented, which traverses the title and description against a comprehensive keyword map of **95+ keyword-to-domain mappings**. The Gemini API calls employ **exponential backoff** (2s → 4s → 8s) with a maximum of 3 retries for rate-limited responses.
- **Skill Extraction:** The system parses unstructured job descriptions to identify and extract up to ten specific professional skills. This is achieved using Gemini AI configured for constrained JSON output generation (`response_mime_type="application/json"`). The rule-based fallback maintains a dictionary of **60+ professional skills** spanning office tools, programming languages, soft skills, and industry-specific competencies.

2.5.3 Phase 3: Deterministic Content Hashing

To facilitate absolute deduplication beyond simple URL collisions, the pipeline generates a deterministic hash identifier for each job. This hash is computed based on the normalized company name, the cleaned job title, and the standardized location, ensuring that identical postings from different source URLs are accurately identified and consolidated.

2.5.4 Phase 4: Persistence

In the final phase, the pipeline assembles the finalized payload, integrating the original fields with the newly computed normalized data and extracted skills. This enriched dataset is then persisted to the PostgreSQL data layer using an upsert operation, updating existing records and inserting new ones seamlessly.

2.5.5 Normalization Flow Diagram

The following diagram visualizes the flow of data through the four phases of the normalization pipeline:

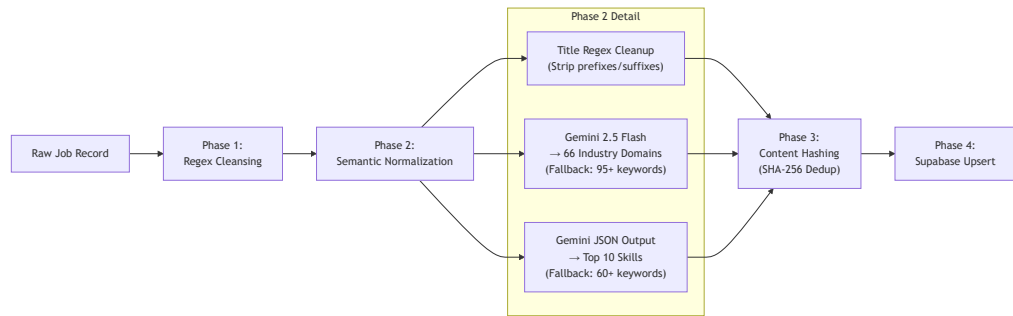


Figure 2.3: Four-Phase Normalization Pipeline

2.6 Key Quantitative Metrics

Metric	Value
Scraper source files	8 files, ~77 KB total
TopCV scraper complexity	323 lines
JobOKO scraper complexity	516 lines
Industry domains (Phase 2)	66 categories
Keyword-to-domain mappings	95+ entries
Skill extraction dictionary	60+ professional skills
CI/CD pipeline timeout	150 minutes
Cron schedule	Every 3 days at 02:00 VN
Concurrency limit (link checking)	10 workers
Batch delete size	50 records
Listing page delay	8,000 ms
Detail page inter-crawl delay	10,000 ms
Gemini retry strategy	3 attempts, exponential backoff (2s, 4s, 8s)

Table 2.2: Data Acquisition Quantitative Metrics

Chapter 3

Polyglot Persistence and Search Infrastructure

3.1 Overview

This chapter outlines the core data architecture of the Job Market Analytics Platform. Managing diverse data types ranging from highly structured user profiles to massive, unstructured job postings requires a specialized approach. The system adopts a polyglot persistence strategy, utilizing five distinct databases (Supabase, Redis, MongoDB, Elasticsearch, and Qdrant), each selected to optimally address specific data shapes and access patterns.

This chapter focuses on the three core platform components: **Supabase** (relational), **Redis** (caching and security), and **Elasticsearch** (full-text search). MongoDB and Qdrant are covered in the companion thesis (Thesis 1) as they primarily serve the AI Chatbot subsystem.

3.2 Polyglot Persistence Justification

The platform handles several distinct data workloads that impose conflicting requirements on a database system:

Workload	Access Pattern	Consistency Requirement
User accounts & profiles	Relational joins, ACID transactions	Strong consistency
Job search queries	Full-text search, faceted aggregations	Eventual consistency
Session tokens & cache	Sub-millisecond reads, automatic expiry	Best-effort
Chat conversation logs	Schema-less document appends	Eventual consistency
Resume embeddings	Approximate nearest neighbor search	Eventual consistency

Table 3.1: Data Workloads and Optimal Databases

3.2.1 Why Not a Monolithic Database?

A traditional, monolithic PostgreSQL approach was initially considered. However, the following trade-offs became evident:

Criteria	Monolithic PostgreSQL	Polyglot Architecture
Full-text search	ILIKE / tsvector limited fuzzy search, no faceted aggregations	Elasticsearch BM25 with native facets
Session caching	Disk-based I/O per session lookup	Redis in-memory at sub-millisecond latency
Vector similarity	pgvector extension single-node, limited indexing algorithms	Qdrant HNSW with named vector support
Schema flexibility	Rigid schema migrations for chat logs	MongoDB schemaless document appends
Operational complexity	Single database, simple operations	5 databases, requires health checks and sync pipelines

Table 3.2: Monolithic vs Polyglot Architecture Trade-offs

The polyglot approach trades operational complexity for optimized performance at each data access layer. The key architectural decision is to designate **Supabase as the single source of truth** and treat all other databases as **read-optimized replicas or caches**, following the Command Query Responsibility Segregation (CQRS) pattern.

3.3 Supabase (Core Relational Database)

Supabase, built on top of PostgreSQL, serves as the primary relational database for the application's core logic.

3.3.1 Structured Entities

Supabase is responsible for managing structured entities that require strong ACID (Atomicity, Consistency, Isolation, Durability) guarantees. The relational model ensures data integrity and enforces complex business rules through foreign key constraints.

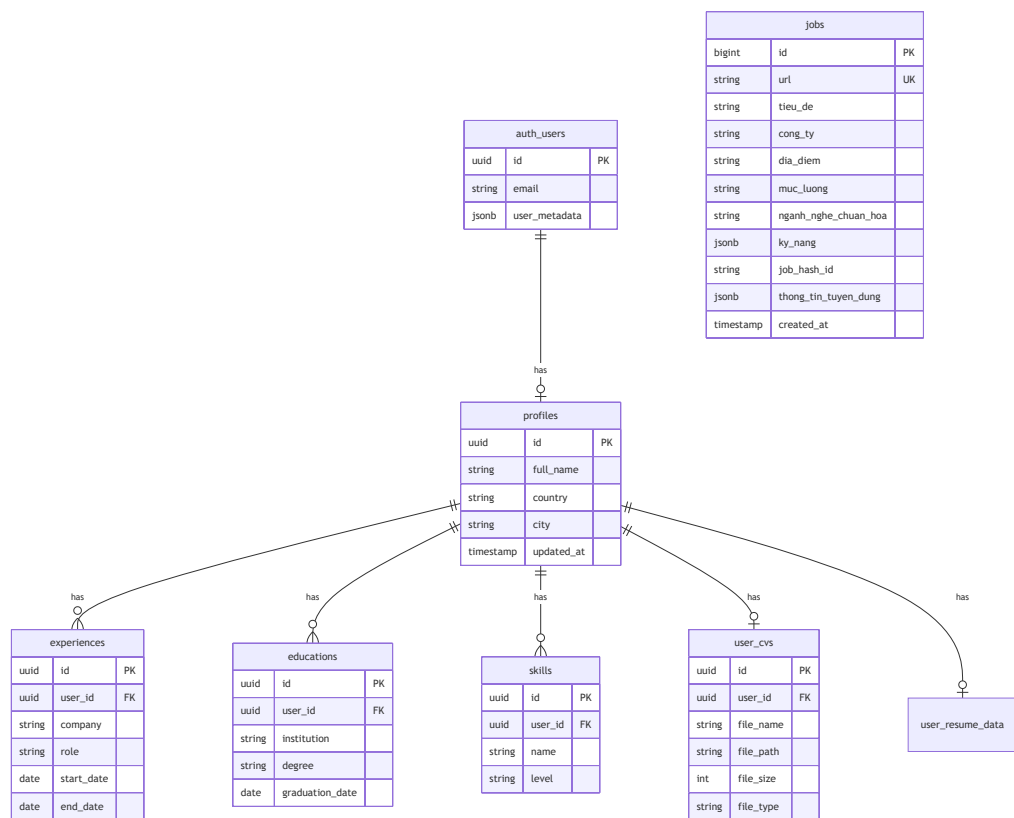


Figure 3.1: Entity Relationship Diagram of Supabase Structured Data

3.3.2 Dual-Client Architecture

The application maintains two distinct Supabase client configurations to enforce access control separation:

- **Browser Client** (@supabase/supabase-js): Uses the public Anon Key, subject to Row Level Security (RLS) policies. Used for user-facing operations.
- **Server Client** (@supabase/ssr): Uses the Service Role Key, bypassing RLS. Used for administrative operations (scraper data ingestion, batch normalization).

3.4 Redis (Session, Cache, and Security Layer)

Redis is deployed as the high-speed, in-memory cache layer, serving three critical roles in the architecture: session management, query caching, and security enforcement.

3.4.1 Singleton Connection Pattern

Within the application, Redis connections are managed via a **global singleton pattern** (`globalForRedis`). This approach prevents connection multiplication during Next.js hot-module-reload (HMR) cycles in development, where module re-imports would otherwise create a new TCP connection per reload. In production (`NODE_ENV=production`), the singleton is not cached globally since HMR is not active.

3.4.2 Redis Key Space Architecture

The codebase employs structured key naming conventions with appropriate TTL strategies:

Key Pattern	Purpose	TTL	Module
<code>rate_limit:{ip}:{window}</code>	Fixed-window rate limiting	<code>windowSeconds + 10</code>	<code>redisSe</code>
<code>blacklist:{sha256_hash}</code>	JWT revocation on logout	Matches JWT <code>exp</code> claim	<code>redisSe</code>
<code>session:{id}</code>	Chatbot session state	24 hours	<code>session</code>
<code>session:{id}:history</code>	Chat history sync buffer	24 hours	<code>session</code>
<code>job:{id}</code>	Async CV processing job status	1 hour	<code>job_tra</code>

Table 3.3: Redis Key Naming Conventions

3.4.3 Core Workloads

- **Authentication Security:** Redis handles both **JWT blacklisting** (SHA-256 hash with TTL matching token expiry) and **rate limiting** (Fixed Window Counter algorithm at 50 requests per 60 seconds per IP). These are detailed in Chapter 5.
- **Job Tracking:** Background processing tasks for CV extraction rely on Redis to track job state and progress with sub-millisecond updates.
- **Enum Caching:** Frequently accessed Elasticsearch aggregation results (cities, experience buckets, work types, categories) are cached in the server's RAM via the `EnumCache` class, with periodic async refresh from Elasticsearch every 3,600 seconds.

3.5 Elasticsearch (Full-Text Search Engine)

To power the core "job search" functionality, the system relies on Elasticsearch 8.13.0. Standard relational databases are not optimized for unstructured text search at scale.

3.5.1 CQRS Data Flow

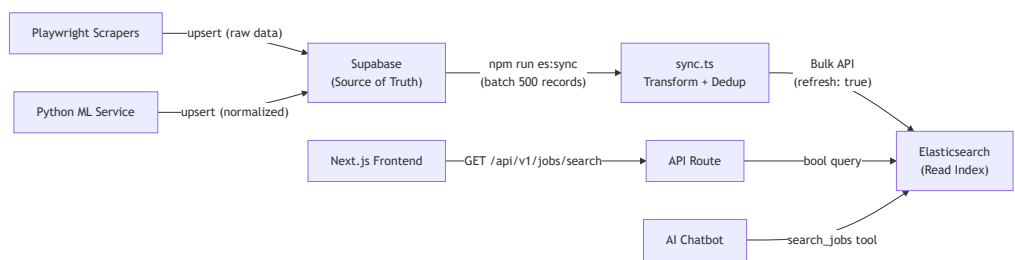


Figure 3.2: CQRS Data Flow to Elasticsearch

3.5.2 Data Synchronization

Job posting data is synchronized from Supabase into Elasticsearch via a batch pipeline (`npm run es:sync`). The sync script employs a **full-rebuild strategy**: it deletes and recreates the entire index on each run, then re-indexes all records. While this approach is less efficient than incremental sync, it guarantees index consistency and eliminates stale-document accumulation. The pipeline processes records in paginated batches of 500, ordered by `created_at` descending (newest first), and applies in-memory URL-based deduplication via a JavaScript `Set`.

3.5.3 Advanced Mapping and Indexing

Elasticsearch indices are carefully mapped to support diverse search requirements:

Field	ES Type	Analyzer	Purpose
<code>url</code>	keyword		Unique document ID
<code>tieu_de</code>	text	standard	Job title full-text search (boosted)
<code>cong_ty</code>	text	standard	Company name search (boosted)
<code>cities</code>	keyword		Location faceted filter (63 provinces)
<code>categories</code>	keyword		Industry domain faceted filter
<code>workTypes</code>	keyword		Work type faceted filter
<code>levels</code>	keyword		Seniority level faceted filter
<code>expBuckets</code>	keyword		Experience tier faceted filter (4 tiers)
<code>salaryBuckets</code>	keyword		Salary tier faceted filter (6 ranges)
<code>created_at</code>	date		Temporal ordering
<code>raw_data</code>	object (enabled: false)		Full job document stored but not indexed

Table 3.4: Elasticsearch Field Mapping Strategy

The `raw_data` field uses `enabled: false` to store the original job JSON without indexing its internal contents. This significantly saves system resources (RAM, CPU, disk) during indexing while retaining all data for retrieval—the frontend reads job details directly from `raw_data` without requiring a secondary query to Supabase.

3.6 Technology Choice: Elasticsearch vs Alternatives

Criteria	Elasticsearch 8.13	Typesense	Meilisearch	PostgreSQL Full-Text
Relevance algorithm	BM25 (default since v5)	TF-IDF variant	Proximity-based	ts_rank with tsvector
Faceted aggregations	Native terms aggregations	Native facets	Native facets	GROUP BY (expensive at scale)
Fuzzy search	Levenshtein edit distance (fuzziness: AUTO)	Typo tolerance (1-2 chars)	Typo tolerance (built-in)	ILIKE / trigram similarity
Vietnamese NLP	Standard analyzer (tokenization, lowercasing)	No built-in CJK support	No built-in CJK support	simple/pg_trgm dictionaries
Deployment	Docker official image	Single binary	Single binary	Built into Supabase
Scalability	Horizontal sharding	Vertical scaling	Vertical scaling	Connection-limited
Ecosystem	Mature (REST API, Kibana)	Lightweight	Lightweight	Native PostgreSQL
Memory usage	~512 MB (JVM heap)	~50 MB	~50 MB	Shared with main DB

Table 3.5: Search Engine Technology Comparison

Elasticsearch was selected for its mature ecosystem, native Docker support, powerful `bool` query DSL, and production-proven aggregation capabilities. Its BM25 relevance scoring provides superior search quality compared to PostgreSQL’s basic full-text capabilities, which is critical for a job search platform processing thousands of Vietnamese-language postings.

3.7 Key Quantitative Metrics

Metric	Value
Total databases in polyglot architecture	5 (Supabase, Redis, Elasticsearch, MongoDB, Qdrant)
Core databases (Thesis 2 scope)	3 (Supabase, Redis, Elasticsearch)
Supabase tables	7 (profiles , experiences , educations , skills , addresses , workplaces , interests)
Redis key patterns	5 distinct naming conventions
Redis rate limit threshold	50 requests / 60 seconds / IP
EnumCache TTL	3,600 seconds (1 hour)
ES index fields	10 mapped fields + 1 stored-only (raw_data)
ES sync batch size	500 records per Supabase fetch
ES JVM heap allocation	512 MB initial / 512 MB maximum
Geographic entities (cities)	63 entries (61 provinces + "Toàn quốc" + "Ngoài nước ngoài")
Industry domain categories	66 standardized categories
Salary buckets	6 ranges (03, 35, 510, 1020, 2050, 50+ triệu VND)
Experience buckets	4 ranges (< 1, 1-2, 3-5, 5+ years)

Table 3.6: Database Infrastructure Metrics

Chapter 4

Full-Text Search Engine (Elasticsearch)

4.1 Search Architecture & Rationale

Building a job search system requires the ability to handle complex text queries, faceted filtering, and fast response times. In the initial phase, a relational database system (PostgreSQL via Supabase) was used as the primary data source. However, as the data volume grew and search requirements became more complex, this architecture revealed several limitations:

- **Limitations of SQL:** Full-text search queries or `ILIKE` operators in PostgreSQL are not optimized for natural language search, especially when handling spelling errors and relevance scoring. PostgreSQL's `tsvector` / `tsquery` system lacks native Vietnamese language support and provides limited relevance tuning compared to specialized search engines.
- **Faceted Aggregations:** Dynamically calculating filter categories (e.g., counting the number of jobs per city or salary range) consumes a lot of resources if executed directly on SQL using complex `GROUP BY` clauses. Each unique filter combination requires a separate query, leading to N+1 aggregation overhead.

To resolve these issues, the system architecture adopts the Command Query Responsibility Segregation (CQRS) pattern:

- **Supabase (PostgreSQL)** acts as the "Source of Truth" (Transactional Database), storing the original data after scraping and normalization.
- **Elasticsearch 8.13** is used as a "Read-only Index" (Analytical Database), dedicated to processing search queries and filtering data for the frontend and AI Chatbot. The Elasticsearch system is deployed as a single-node cluster via Docker (`elasticsearch:8.13.0` on port 9200), ensuring lightning-fast search capabilities and multi-language support.

4.2 Index Schema & Mapping Strategies

To optimize Elasticsearch for both relevance scoring and exact filtering, the index structure (`jobs`) is meticulously designed within the data synchronization configuration file.

Two main data type strategies are used:

1. **Keyword Mapping:** Applied to fields that require exact match filtering and aggregations, such as `url`, `cities` (63 geographic entities), `categories` (66 industry domains), `workTypes`, `levels`, `expBuckets` (4 tiers), and `salaryBuckets` (6 ranges). The `keyword` type allows Elasticsearch to build optimal data structures (inverted index on exact terms) for directly searching original text structures without tokenization.
2. **Text Mapping:** Applied to fields that require relevance scoring, such as `tieu_de` (job title) and `cong_ty` (company name), using the default `standard` analyzer. The standard analyzer performs Unicode-aware tokenization, lowercasing, and stop word removal, enabling BM25-based relevance ranking.

Optimizing the "Document Store" with `enabled: false`: Instead of extracting every detail of the job for indexing, the `raw_data` field is configured as an `object` type with the `enabled: false` option. This option instructs Elasticsearch to store the original JSON string of the job without indexing its internal contents. This approach significantly saves system resources (RAM, CPU, disk space) during the indexing process while retaining all the data. When a search query returns results, the frontend can immediately retrieve detailed job data from `raw_data` without needing an additional query to Supabase for the information.

4.3 Data Ingestion & Deduplication Pipeline (Sync Flow)

Data is updated from Supabase to Elasticsearch via a batch synchronization pipeline. This process is triggered by a Node.js script (`sync.ts`).

The sync pipeline employs a **full-rebuild strategy**: the existing index is deleted and recreated on each run, ensuring zero stale-document accumulation. While this is less efficient than incremental sync for very large datasets, it guarantees index consistency and eliminates the need for complex change-detection logic.

Figure 4.1: Elasticsearch Batch Synchronization Pipeline

1. **Batch Fetching:** The system fetches data from Supabase in paginated batches (500 records per batch) ordered by `created_at` descending (newest first). Breaking the data down into batches helps prevent memory overflow.
2. **Memory-Resident Deduplication:** To avoid data duplication before pushing to Elasticsearch, an in-memory `Set` data structure (RAM) is used to track records that have already appeared. The primary deduplication key is the job URL. If the URL is missing, a composite key in the format `${tieu_de}-${cong_ty}` (title-company) is used as the fallback identifier.
3. **Normalization Transform:** Each job record passes through five normalization functions (`splitLocations`, `getSalaryBuckets`, `getExpBuckets`, `getWorkTypeTags`, `getLevels`) that convert unstructured Vietnamese text into standardized keyword facets.
4. **Bulk API:** The deduplicated and normalized data is pushed using the `_bulk` API, which exponentially improves write speeds compared to sequentially writing individual documents. These operations are executed with the `refresh: true` option to ensure that new data is searchable immediately after the sync completes.

4.4 Vietnamese NLP & Heuristic Normalization Logic

Due to the unstructured nature of Vietnamese job postings, an internal library (`helpers.ts`, 152 lines) was built to extract and normalize data through Heuristic and Regex techniques prior to indexing:

- **Location Parsing (`splitLocations`):** Raw location strings (e.g., "Ni làm vic: Hà Ni, Tp. HCM") are stripped of unnecessary prefixes, split by commas, and matched against a static array of **63 provinces/cities** (`CITY_PATTERNS`). The system applies Regex for noise filtering when job title keywords (such as "chuyên viên", "trng phòng") are mistakenly identified as locations. Notably, the parser implements a **negative lookbehind regex** `(?<!Bà Ra) - (?!Vng Tàu)` to prevent incorrectly splitting the compound province name "Bà Ra - Vng Tàu"

on the hyphen delimiter a critical edge case in Vietnamese geographic parsing.

- **Salary Bucketing (`getSalaryBuckets`):** Foreign currency salaries are filtered out using a comprehensive regex matching **17 international currency codes** (USD, EUR, GBP, JPY, SGD, AUD, CAD, HKD, KRW, THB, MYR, INR, CNY, RMB, TWD, CZK, CHF). For VND salaries, the system uses Regex to extract number ranges (e.g., "10 - 15 triu"), removes commas, converts them to floating-point decimals representing millions of VND, and classifies them into 6 fixed income ranges: 03, 35, 510, 1020, 2050, and over 50 million VND.
- **Experience Parsing (`getExpBuckets`):** Text regarding experience (e.g., "di 1 nm", "trên 5 nm", or ranges like "1 - 2 nm") is scanned and normalized into a time range value in years. Month-based values (e.g., "2-5 tháng") are automatically converted to years by dividing by 12. It is then assigned to 4 static experience buckets: Under 1 year, 12 years, 25 years, Over 5 years.
- **Work Types (`getWorkTypeTags`):** English slang or mixed Vietnamese keywords (e.g., "full-time", "part-time", "thi v", "wfh", "hybrid") are mapped to 5 standard tags: "Toàn thời gian" (Full-time), "Bán thời gian" (Part-time), "Thực tập" (Internship), "Thị v" (Contract), "Làm từ nhà" (Remote/WFH).

4.5 Multi-Field Search & Relevance Tuning

When a search command is received from the AI Chatbot or the frontend API, the query is constructed via the Python async Elasticsearch client (`data_clients.py`) using the `bool` Query DSL.

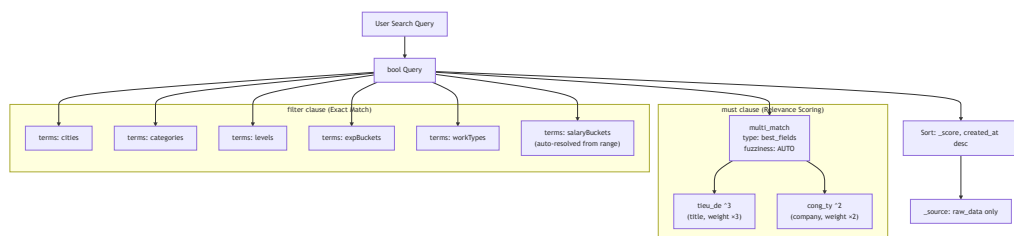


Figure 4.2: Elasticsearch bool Query Construction

1. **Relevance Scoring (Must Clause):** If the user provides a **keyword**, the query uses the `multi_match` command with the `best_fields` option. The job title (`tieu_de`) is weighted with a multiplier of 3 (~ 3), while the company name (`cong_ty`) receives a weight multiplier of 2 (~ 2). The `fuzziness: AUTO` feature enables Levenshtein edit-distance tolerance, allowing 0 edits for terms 12 characters, 1 edit for 35 characters, and 2 edits for 6+ characters. This handles common typos but is not Vietnamese-specific NLP – it operates at the character level.
2. **Strict Filtering (Filter Clauses):** Optional parameters such as location, salary, and experience are passed in as `terms` filters, instructing the engine to strictly match the content against the defined **keyword** types. Filter clauses do not affect relevance scoring and are cached by Elasticsearch for performance.
3. **Salary Resolution:** When given an input range for desired salary (`min_salary` to `max_salary`), the algorithm automatically identifies all salary buckets that overlap with the request and injects them into the query as a list of valid keywords. For example, a request for 815 million VND would resolve to buckets "5 10 triu" and "10 20 triu".
4. **City Prioritization:** If a user searches by location, the returned payload within `raw_data` is automatically adjusted by moving the queried city to the top of the `cities` list. This reordering helps the frontend interface and AI better assess visual priority when displaying multiple locations.
5. **Source Filtering:** The `_source` field is restricted to `["raw_data"]` only, avoiding the transfer of redundant indexed fields in the response payload.

4.6 Aggregation & Memory-Resident Enum Caching

To build a UI filtering experience or to provide a valid set of parameters for the AI Chatbot's Tool Calling process, the system needs to know the available filter values (e.g., which cities currently have job postings).

- **Elasticsearch Aggregations:** The client sends queries with a size of 0 (`size: 0`) requesting `terms aggregations` to group distinct attributes (`distinct_cities` up to 1,000 buckets, `distinct_categories` up to 500 buckets).
- **EnumCache Pattern:** To avoid constantly sending Aggregation queries that consume Elasticsearch resources, the `EnumCache` class (`enum_cache.py`, 138 lines) maintains these lists directly in the server's memory (RAM) with a Time-To-Live (TTL) of 3,600 seconds (1 hour).
- **Async Background Refresh:** The cache refresh process runs as a background task via `asyncio.Task` (`_refresh_loop`). Therefore, any GET request from the frontend or Pydantic Validator from the Chatbot can retrieve these data tags in sub-millisecond times thanks to synchronous properties (`@property`), significantly improving the application's overall response speed.

4.7 Key Quantitative Metrics

Metric	Value
ES Docker image version	<code>elasticsearch:8.13.0</code>
JVM heap allocation	512 MB initial / 512 MB maximum
Sync batch size	500 records per Supabase fetch
City patterns dictionary	63 geographic entities
Industry domain categories	66 standardized tags
Salary buckets	6 ranges (VND millions)
Experience buckets	4 ranges
Work type tags	5 standard categories
Foreign currency filter	17 international currency codes
Title boost weight	$\times 3$ (<code>tieu_de^3</code>)
Company boost weight	$\times 2$ (<code>cong_ty^2</code>)
EnumCache TTL	3,600 seconds (1 hour)
Max aggregation buckets (cities)	1,000
Max aggregation buckets (categories)	500
helpers.ts complexity	152 lines
enum_cache.py complexity	138 lines

Table 4.1: Elasticsearch Infrastructure Metrics

Chapter 5

Authentication & Authorization

5.1 Architecture Overview

In the CareerIntel platform, securing user data and managing access control are pivotal. Instead of building and maintaining a custom JSON Web Token (JWT) issuance system from scratch, the project leverages a managed authentication model via **Supabase Auth**. Supabase provides a robust identity solution, backed by PostgreSQL, which integrates tightly with the Next.js 16 Middleware architecture.

To add an additional security layer, the platform integrates **Redis** to handle Token Revocation (Blacklisting) immediately when a user logs out, and **Rate Limiting** to protect API endpoints from abuse. The combination of Supabase (for identity verification) and Redis (for session state control and security enforcement) creates a system that is both secure and highly flexible.

5.1.1 Technology Choice: Supabase Auth vs Alternatives

Criteria	Supabase Auth	Auth0	Firebase Auth	Custom JWT
Hosting	Managed (PostgreSQL-backed)	Managed SaaS	Managed (Google Cloud)	Self-hosted
Pricing	Free tier (50K MAU)	Free tier (7.5K MAU)	Free tier (10K verifications/month)	Infrastructure cost only
Next.js integration	@supabase/ssr with cookie management	@auth0/nextjs-auth0	firebase-admin SDK	Manual cookie/header management
Database integration	Native PostgreSQL RLS policies	External user store	Firestore integration	Custom user tables
Social login	Google, GitHub, OAuth	30+ social providers	Google, Facebook, Twitter	Manual OAuth implementation
JWT customization	Limited (via PostgreSQL hooks)	Extensive (Actions/Rules)	Limited	Full control
RLS (Row Level Security)	Native support	Not applicable	Firestore Security Rules	Manual implementation
Operational overhead	Minimal	Minimal	Minimal	High (key rotation, revocation, migration)

Table 5.1: Authentication Technology Comparison

Supabase Auth was selected for its **native PostgreSQL Row Level Security (RLS) integration**, which allows database-level access control policies without custom middleware. Since the application already uses Supabase for its relational data layer, using its built-in auth system eliminates the need for a separate identity provider and reduces integration complexity.

5.2 Authentication Flow

The authentication process is designed around Server-side Cookies rather than storing tokens in the browser's Local Storage. This approach helps mitigate Cross-Site Scripting (XSS) attacks aimed at token theft.

1. **Login:** The user provides an email and password. The frontend calls a Server Action (utilizing Supabase's `signInWithPassword`).
2. **Token Issuance:** Supabase verifies the credentials and returns a JWT. This JWT is set into an HTTP-only Cookie by the Server Client.
3. **Request Validation:** Whenever the browser makes a request to protected pages, the Next.js Middleware reads the Cookie and validates the JWT using `supabase.auth.getUser()`.
4. **Logout:** When the user logs out, the current JWT is extracted and hashed, then stored in the Redis Blacklist with a Time-To-Live (TTL) corresponding to the token's remaining expiration time. Finally, the Supabase `signOut` function is called to clear the Cookie.

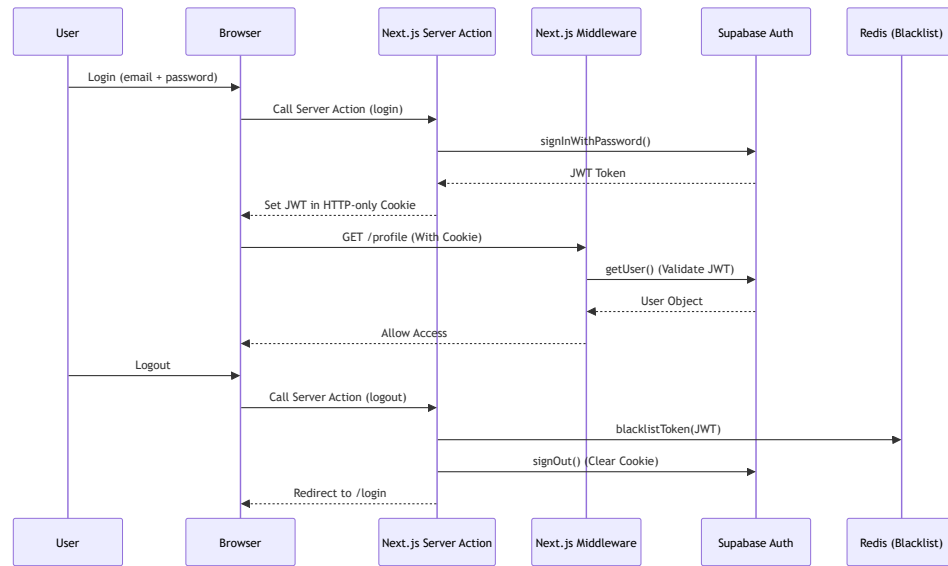


Figure 5.1: Authentication and Logout Flow

5.3 Route Protection Strategy

The platform's routing system is divided into two primary groups: Public Routes and Protected Routes. Access control is enforced in the `middleware.ts` file using the `createServerClient` from the `@supabase/ssr` library. The middleware intercepts every incoming request and validates the JWT before allowing access.

- **Protected Pages:** Pages such as `/profile`, `/ai`, `/search`, `/insights`, and `/job` (including all nested routes such as `/job/[id]`) strictly require a valid session. The `startsWith()` matching ensures all child routes inherit the protection policy.
- **Public vs Protected APIs:** Certain APIs like `/api/chatbot` and `/api/kie` are publicly accessible, whereas all other `/api/*` routes require authentication.

Handling Unauthenticated Requests: The system handles requests intelligently based on their context.

- If a user attempts to access a protected page (UI) without a token, the Middleware responds with a `307 Redirect` to the `/login` page.
- However, if a request is made to a Protected API (for example, fetching personal data via a `fetch` call), a redirect is inappropriate and would cause client-side errors. In this scenario, the Middleware returns a `401 Unauthorized` status code in JSON format, allowing the frontend to catch and handle the error gracefully.

5.4 Advanced Security: JWT Blacklisting via Redis

An inherent drawback of Stateless JWT architecture is that a token cannot be revoked before its expiration time. To address this, the project implements a **JWT Blacklisting** mechanism using Redis within the `redisSecurity.ts` module.

5.4.1 Optimized Storage Process:

Instead of storing the raw JWT string (which may contain sensitive payloads and consume more storage), the system employs the **SHA-256** algorithm to generate a Hash from the JWT via the `getTokenHash()` function. This hash serves as the key stored in Redis. Furthermore, the key's Time-To-Live (TTL) in Redis is calculated based on the JWT's `exp` claim, by subtracting the current time from the expiration time (`getJwtExpiry()`). Consequently, Redis automatically frees up memory once the token expires, eliminating the need for manual garbage collection.

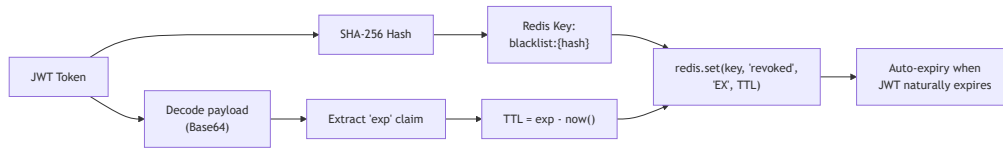


Figure 5.2: JWT Blacklisting Storage Process

5.4.2 Fail-Closed vs Fail-Open: Deliberate Security Trade-offs

The `redisSecurity.ts` module implements two distinct failure strategies depending on the security criticality of the operation:

Function	Failure Strategy	Rationale
<code>isTokenBlacklisted()</code>	Fail-Closed (treat as blacklisted)	Security > Availability provoked tokens from slipping Redis is unreachable
<code>checkRateLimit()</code>	Fail-Open (allow request through)	Availability > Security ing failure should not block search requests

Table 5.2: Failure Strategies in Redis Security Module

This deliberate contrast ensures that authentication-critical operations prioritize security (forcing affected users to re-login temporarily), while non-critical operations like rate limiting prioritize availability (allowing requests through with degraded protection).

5.5 Rate Limiting

In addition to JWT blacklisting, the `redisSecurity.ts` module implements **API Rate Limiting** using the **Fixed Window Counter** algorithm.

5.5.1 Mechanism:

1. A Redis key is constructed from the client's IP address and the current time window: `rate_limit:{ip}:{floor(now / windowSeconds)}`.
2. Each incoming request atomically increments the key using `redis.incr(key)`.
3. If the key is newly created (count = 1), a TTL of `windowSeconds + 10` seconds is set to ensure automatic cleanup.
4. If the count exceeds the configured limit (default: **50 requests per 60 seconds**), the request is rejected.

5.5.2 Design Decisions:

- **Fixed Window vs Sliding Window:** The Fixed Window algorithm was chosen for its simplicity and minimal Redis operations (single `INCR` per request). While it can allow up to $2\times$ burst at window boundaries (a known limitation), this trade-off is acceptable for the platform's traffic patterns.
- **TTL Buffer (+10s):** The extra 10 seconds on the TTL ensures that the Redis key outlives the window, preventing a race condition where the key expires mid-window and resets the counter.

5.6 Form Management and Input Validation

The processing of authentication forms (Login, Signup, Profile Update) is handled through the **Next.js Server Actions** (`'use server'`) paradigm. This approach completely eliminates the need to create intermediary API routes solely for receiving form data, thereby reducing network complexity and improving processing speed.

All input data is strictly validated using the **Zod** library (`LoginSchema`, `SignupSchema`). Actions that record profile data (Experience, Education, Skill, CV) into public PostgreSQL tables (via the Supabase client) are tightly bound to the current `user_id`. This ensures data integrity and prevents Insecure Direct Object Reference (IDOR) vulnerabilities—a user can only create, update, or delete records where `user_id` matches their authenticated session.

5.7 Key Quantitative Metrics

Metric	Value
Protected page routes	5 (/profile, /ai, /search, /insights, /job)
Public API routes	2 (/api/chatbot, /api/kie)
Rate limit threshold	50 requests / 60 seconds / IP
Rate limit algorithm	Fixed Window Counter
JWT hash algorithm	SHA-256
Blacklist TTL	Dynamic (matches JWT exp claim)
Default fallback TTL	3,600 seconds (1 hour)
Rate limit TTL buffer	+10 seconds beyond window
Auth validation schemas	2 (LoginSchema, SignupSchema)
Profile CRUD tables	5 (profiles, experiences, educations, skills, user_cvs)
redisSecurity.ts complexity	126 lines
middleware.ts complexity	59 lines
actions.ts complexity	297 lines (8 Server Actions)

Table 5.3: Authentication and Security Metrics

Chapter 6

Core Frontend Layer (Next.js)

6.1 Overview

The core frontend of the CareerIntel platform is constructed using the Next.js 16 App Router architecture. This layer serves as the primary gateway for users, offering a cohesive application shell, robust job search interfaces, and comprehensive profile management tools. This chapter details the foundational frontend architecture, focusing on server-side rendering strategies for authentication, the URL-driven state of faceted search interfaces, and the offloading of Vietnamese Natural Language Processing (NLP) to the client.

6.1.1 Technology Choice: Next.js App Router vs Alternatives

Criteria	Next.js App Router	Next.js Pages Router	React + Vite (SPA)	Nuxt.js (Vue)
Rendering model	Hybrid SSR + Client	SSR via <code>getServerSideProps</code>	Client-only (CSR)	Hybrid SSR + Client
Server Components	Native (React Server Components)	Not supported	Not applicable	Not supported
Auth data fetching	Server-side cookie access in RSC	<code>getServerSideProps</code> only	Client-side only (flash of unauth content)	<code>useAsyncData</code> in server middleware
Route protection	<code>middleware.ts</code> edge function	<code>_middleware.ts</code> (experimental)	Manual route guards	<code>definePageMeta</code> middleware
API layer	File-based API routes in <code>app/api/</code>	File-based in <code>pages/api/</code>	Separate backend required	<code>server/api/</code> directory
Server Actions	Native <code>'use server'</code> for forms	Not supported	Not applicable	<code>useAsyncData</code> with API calls
Streaming/Suspense	<code>Suspense</code> <code>loading.tsx</code> + <code>Suspense</code>	Manual implementation	Not applicable	<code><Suspense></code> support
Bundle splitting	Automatic per-route	Manual dynamic imports	Manual lazy imports	Automatic per-route

Table 6.1: Frontend Architecture Technology Comparison

Next.js App Router was selected for its native Server Components support, which enables secure server-side authentication cookie handling, eliminates the "flash of unauthenticated content" common in SPAs, and provides built-in Server Actions that remove the need for intermediary API routes for form

submissions.

6.2 SSR Data Fetching and User-Aware Rendering

Next.js 16 introduces a clear delineation between Server Components and Client Components. The application leverages Server Components as the default execution environment, heavily optimizing Time to Interactive (TTI) and securing the authentication flow.

To render personalized content based on the user's authentication state, Server-Side Rendering (SSR) data fetching is deeply integrated with Supabase. The application utilizes a server-side Supabase client instantiated via `createClient()`. This client intercepts the incoming HTTP request in the Node.js runtime, reads the secure, HttpOnly JWT cookies, and validates the user session before emitting any HTML to the browser. This architectural choice allows protected routes, such as the `/profile` page, to be fully hydrated with the user's specific data (experiences, education, skills) on the server. This eliminates the "flash of unauthenticated content" typical of Single Page Applications (SPAs) and enhances security by preventing client-side interception of raw tokens.

6.3 Faceted Search Architecture

The job search interface, located at `/search`, provides users with an Elasticsearch-powered faceted search experience. Managing the complex state of numerous interdependent filters requires a robust state management strategy.

6.3.1 URL-Driven State Management

Instead of relying on isolated React state (`useState` or `Redux`), the search interface anchors its state entirely to the browser's URL query parameters via Next.js's `useSearchParams()`. When a user interacts with the custom `DropdownFilter` component (e.g., selecting "Ho Chi Minh City" and "Senior Level"), the client component immediately updates the URL. This mutation triggers a shallow route update, prompting a `useEffect` hook to serialize the parameters and invoke the `/api/v1/jobs/search` API endpoint.

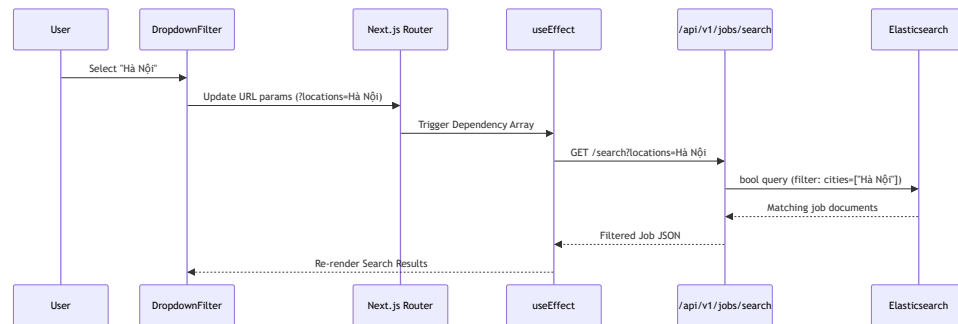


Figure 6.1: URL-Driven Search State Flow

This architecture ensures that complex search queries are entirely linkable, shareable, and resilient to page reloads, maintaining a stateless and predictable data flow.

6.3.2 Supported Filter Parameters

URL Parameter	ES Field	Type	Example
q	tieu_de, cong_ty	multi_match (keyword)	?q=developer
locations	cities	terms filter	?locations=Hà Ni
categories	categories	terms filter	?categories=IT Phn mm
levels	levels	terms filter	?levels=Qun lý
expBuckets	expBuckets	terms filter	?expBuckets=2 5 nm
salaryBuckets	salaryBuckets	terms filter	?salaryBuckets=10 20 triu
workTypes	workTypes	terms filter	?workTypes=Toàn thi gian
page	ES from	Pagination offset	?page=2

Table 6.2: Supported Filter Parameters for Faceted Search

6.4 Client-Side Vietnamese NLP Processing

While the core Machine Learning normalization pipeline processes job data asynchronously on the backend, the frontend implements lightweight, deterministic Natural Language Processing (NLP) tailored for the Vietnamese language. This offloading strategy reduces API overhead and allows for instantaneous client-side formatting and data binning.

6.4.1 Location and Geographic Parsing (`splitLocations`)

Job postings on Vietnamese platforms frequently concatenate multiple geographic regions into unstructured strings (e.g., "Khu vc: H Chí Minh, Hà Ni - à Nng"). The frontend implements a `splitLocations` parser utilizing a predefined dictionary of **63 valid geographic entities** (61 Vietnamese provinces/cities plus "Toàn quc" and "Nc ngoài") and localized regex patterns.

The parser performs the following operations:

1. **Prefix stripping:** Removes common Vietnamese location prefixes ("Ni làm vic:", "Khu vc:", "Ti:").

2. **Intelligent delimiter splitting:** Splits on commas and semicolons. For hyphens, a **negative lookbehind regex** (`(?<!Bà Ra) - (?!Vng Tàu)`) is employed to prevent incorrectly splitting the compound province "Bà Ra - Vng Tàu" a critical Vietnamese geographic edge case where the dash is part of the official name.
3. **Dictionary matching:** Each fragment is matched against `CITY_PATTERNS`. The `normalizeLocation` function additionally filters out false positives by rejecting fragments that match job title keywords (e.g., "chuyên viên", "trng phòng", "giám c") using a Vietnamese occupational regex filter.

6.4.2 Unstructured Salary Categorization (`getSalaryBuckets`)

Salary information poses a significant parsing challenge, often appearing as free text such as "15 - 20 Triu", "Lên n 1000\$", or "Tho thun". The frontend utilizes a `getSalaryBuckets` function executing heuristic regex matching.

The parser first filters out **17 foreign currency codes** (USD, EUR, GBP, JPY, SGD, AUD, CAD, HKD, KRW, THB, MYR, INR, CNY, RMB, TWD, CZK, CHF) via regex to exclude non-VND salaries. It then identifies numeric ranges, standardizes values into millions of VND, and mathematically evaluates the bounds (`[lo, hi]`) against 6 predefined salary tiers. This transforms free text into indexable categorizations like "10 20 triu" without requiring a backend round-trip.

6.4.3 Experience Tier Mapping (`getExpBuckets`)

The `getExpBuckets` parser processes strings like "Di 1 nm", "trên 5 nm", or "2-5 tháng". It handles four Vietnamese expression patterns:

1. **"di" (under):** e.g., "di 1 nm" $\rightarrow$ range `[0, 0.99]`
2. **"trên/hn/ít nht" (over/more than/at least):** e.g., "trên 5 nm" $\rightarrow$ range `[5.01, ∞]`
3. **Range expressions:** e.g., "1 - 2 nm" $\rightarrow$ range `[1, 2]`
4. **Single values:** e.g., "3 nm" $\rightarrow$ range `[3, 3]`

It converts all parsed numeric values into a standardized yearly metric (dividing month-based values by 12) and determines intersection overlaps with 4 predefined experience tiers ("Di 1 nm", "1 2 nm", "2 5 nm", "Trên 5 nm"). By executing these deterministic parsers directly in the browser's JavaScript engine, the frontend remains highly responsive while ensuring data conforms strictly to the schema expected by the Elasticsearch backend.

6.5 Page Architecture

Route	Server/Client	Key Features
/ (Home)	Server + Client	Hero section, search bar with drop-down filters, feature cards
/search	Client	ES-powered faceted search with 7 filters, pagination, URL-driven state
/profile	Server + Client	Profile CRUD (experience, education, skills, CV upload via Supabase Storage)
/insights	Server + Client	Market analytics dashboard
/login	Client	Login form with Zod validation, Server Action submission
/signup	Client	Registration form with email confirmation flow
/job/[id]	Server	Individual job detail page (dynamic route)

Table 6.3: Next.js Application Page Architecture

6.6 Key Quantitative Metrics

Metric	Value
Total page routes	7 distinct pages
API routes	9 route handlers
Geographic entities in dictionary	63 (61 provinces + 2 meta-regions)
Foreign currency codes filtered	17
Salary buckets	6 ranges
Experience buckets	4 ranges
Work type categories	5 standard tags
Vietnamese expression patterns (exp)	4 distinct regex patterns
Shared components	2 (<code>Navbar.tsx</code> , <code>RequireLogin.tsx</code>)
helpers.ts shared parser	152 lines

Table 6.4: Frontend Architecture Quantitative Metrics